SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: OPERATING SYSTEMS COURSE CODE: CSA04

COURSE OUTCOME COVERED

CO2: Analyze process management and deadlock handling techniques to improve CPU utilization and system performance(BL3-BL4)

Assessment Tool Weightage: 30%

QUESTIONS: 10 **Total Marks:50**

Annexure A

Scenario based

Code of Conduct:

*I **G. Santosh Kumar** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: G. Santosh Kumar

Question 1 — CPU Scheduling in a Real-Time Ticket Booking System

A railway ticket booking system experiences heavy load during festival seasons. The system handles three categories of processes:

- P1 – Payment gateway transactions (High priority)
- P2 – Seat availability checks (Medium priority)
- P3 – Ticket confirmation and printing (Low priority)
- Burst times vary from 2 ms to 20 ms.

Recently, customers complained that ticket confirmations are taking too long.

Tasks

1. Identify the most suitable scheduling algorithm (FCFS, SJF, Priority, RR) for this scenario.
2. Justify *why* your chosen method improves CPU utilization and reduces customer wait time.
3. Provide a sample execution order for the above categories.
4. Discuss one drawback of this scheduling method in real-time systems.

(10 Marks)

Question 2 — Deadlock in a Banking Transaction Server

A bank server handles the following operations simultaneously:

- T1: Transfers money between accounts
- T2: Updates passbook entries
- T3: Generates monthly statements

Due to simultaneous locking of Account Database (A), Customer Info Database (B), and Transaction Logs (C), the system enters a state where:

- T1 holds A and requests B
- T2 holds B and requests C
- T3 holds C and requests A

Tasks

1. Draw the resource request situation (text-based RAG).
2. Identify whether a deadlock has occurred and justify your answer.
3. Recommend *one specific technique* (avoidance/prevention/recovery) and explain how it fixes the issue.
4. Suggest a redesign for the locking order to avoid future deadlocks.

(10 Marks)

Question 3 — Mobile OS App Switching Lag

A mobile operating system is designed to support smooth app switching.

But users report that:

- Opening WhatsApp delays the background music player.
- Switching between social media apps feels laggy.
- CPU shows high idle times despite multiple running apps.

The OS currently uses Round Robin (quantum = 20 ms) with 20 background apps active.

Tasks

1. Analyze why the CPU remains idle despite many active processes.
2. Determine whether the time quantum is helping or hurting performance.
3. Recommend a better scheduling method or quantum value, with justification.
4. Explain how the change will reduce switching lag and improve system throughput.

(10 Marks)

Question 4 — Deadlock Risk in an Operating System Update

During a Windows OS update, several modules run in parallel:

- M1: Copy update files
- M2: Install drivers
- M3: Update registry
- M4: Validate system integrity

Resources involved:

- R1 = Disk I/O
- R2 = Kernel lock
- R3 = Configuration lock

Recently logs show:

- M1 holds R1, waiting for R2
- M2 holds R2, waiting for R3
- M3 holds R3, waiting for R1

Tasks

1. Identify whether this sequence can lead to a deadlock.
2. If yes, explain the exact circular wait condition.
3. Propose *Banker's algorithm style* resource allocation to avoid risk.

4. Suggest how OS designers can prioritize modules to improve CPU utilization during update.

(10 Marks)

Question 5 — Cloud Server CPU Utilization Analysis

A cloud hosting provider notices:

- 80% of the time, VM processes wait for network or disk
- Only 20% of the time is spent on CPU
- Each server runs 7 VMs

Using the formula:

$$U = 1 - p^n$$

where p = I/O wait probability

Tasks

1. Calculate CPU utilization.
2. Interpret whether the server is underutilized or optimally loaded.
3. Suggest how increasing or decreasing number of VMs affects throughput.
4. Propose two OS-level strategies for improving CPU utilization in this cloud environment.

(10 Marks)

Question 6 — Hospital Emergency System Process Scheduling

A hospital information system manages:
• P1 – Emergency patient registration (Highest Priority)
• P2 – Laboratory report generation (Medium Priority)
• P3 – Pharmacy billing (Low Priority)

During peak hours, pharmacy billing is delayed while emergency requests continue to arrive.

Tasks
1. Identify the scheduling issue occurring in the system.
2. Recommend a suitable CPU scheduling algorithm.
3. Explain how starvation can be prevented.
4. Discuss how your solution improves response time and fairness.

(10 Marks)

Question 7 — E-Commerce Warehouse Synchronization

An e-commerce warehouse has multiple worker threads updating a shared inventory database simultaneously.

Developers observe:
• Incorrect stock counts
• Duplicate order confirmations
• Frequent data inconsistencies

Tasks
1. Identify the synchronization problem.
2. Explain why a critical section is required.
3. Recommend either semaphores or monitors to solve the issue.
4. Explain how the proposed solution maintains data consistency.

(10 Marks)

Question 8 — Video Streaming Service Thread Management

A video streaming platform creates one thread for every client connection.

During a live sports event:

• 60,000 users connect simultaneously.
• CPU utilisation reaches 95%.
• Memory consumption increases rapidly.
• Video buffering becomes frequent.

Tasks
1. Identify the thread-management problem.
2. Recommend a suitable multithreading model.
3. Suggest two optimisations to improve scalability.
4. Explain how your recommendations improve throughput.

(10 Marks)

Question 9 — Smart Traffic Control Deadlock
A smart city traffic management system controls four intersections.

Resources:
• R1 – North Junction
• R2 – East Junction
• R3 – South Junction
• R4 – West Junction

Current situation:
• T1 holds R1 and requests R2.
• T2 holds R2 and requests R3.
• T3 holds R3 and requests R4.
• T4 holds R4 and requests R1.

Tasks
1. Determine whether a deadlock exists.
2. Explain the four necessary conditions for deadlock in this scenario.
3. Recommend one recovery technique.
4. Suggest a redesign that prevents similar deadlocks.

(10 Marks)

Question 10 — University Examination Server Resource Allocation
An online examination server allocates CPU, memory and database connections to student sessions.

At examination time:
• Thousands of students log in simultaneously.
• New resource requests arrive continuously.
• The administrator wants to avoid deadlocks without reducing resource utilisation.

Tasks
1. Explain how Banker's Algorithm can be applied.
2. Describe the information required before granting a request.
3. Explain how the system determines whether it is in a safe state.
4. Discuss one limitation of using Banker's Algorithm in large-scale cloud systems.

(10 Marks)

Question 1 — CPU Scheduling in a Real-Time Ticket Booking System

1. Understand the Scenario

Three process categories compete for CPU: P1 (payment, high priority), P2 (seat availability check, medium priority), P3 (ticket confirmation/printing, low priority). Burst times range 2–20 ms. Ticket confirmations (P3) are getting delayed.

2. Identify the OS Concept

Concept: Priority-based CPU Scheduling (preemptive), since processes have distinct, business-defined priority levels.

3. Analysis

Priority Scheduling always dispatches the highest-priority ready process. Since payment (P1) and seat-checks (P2) arrive continuously during heavy load, low-priority P3 keeps getting pushed back — this explains the reported confirmation delay.

4. Step-by-Step Solution

Sample execution order (illustrative burst times assumed within given 2–20 ms range: P1 = 5 ms, P2 = 10 ms, P3 = 20 ms, all arrive at t = 0):

Process	Priority	Burst Time (ms)	Order
P1	High	5	1st
P2	Medium	10	2nd
P3	Low	20	3rd

```
Gantt Chart:
 |--- P1 ---|------ P2 ------|---------- P3 ----------|
 0          5                15                       35
```

Process	Waiting Time	Turnaround Time
P1	0	5
P2	5	15
P3	15	35

5. Justification

• Payment transactions (revenue-critical, time-sensitive) execute first — reduces failed/timed-out payments.

• Seat availability runs next, keeping booking flow responsive.

• CPU is never idle while higher-priority work is pending → CPU utilization improves.

Drawback: Strict priority scheduling can cause starvation — if P1/P2 arrive continuously, P3 (confirmation/printing) may be delayed indefinitely, matching the reported complaint.

Final Answer

Preemptive Priority Scheduling is the most suitable algorithm. It should be combined with aging (gradually raising P3's priority the longer it waits) to prevent starvation while still favouring payment and seat-check operations.

Question 2 — Deadlock in a Banking Transaction Server

1. Understand the Scenario

T1 holds A, requests B. T2 holds B, requests C. T3 holds C, requests A. Resources: Account DB (A), Customer Info DB (B), Transaction Logs (C).

2. Identify the OS Concept

Concept: Deadlock via circular wait among three transactions holding mutually exclusive locks.

3. Analysis — Resource Allocation Graph

```
T1 --> B      (T1 holds A, requests B)
^        |
|        v
A        T2 --> C   (T2 holds B, requests C)
^                |
|                v
T3 <------------ C   (T3 holds C, requests A)

Cycle: T1 -> B -> T2 -> C -> T3 -> A -> T1
```

4. Step-by-Step Solution

Deadlock confirmed: Yes. A closed cycle T1→B→T2→C→T3→A→T1 exists, and each resource (A, B, C) has exactly one instance, so the cycle guarantees deadlock.

Relevant Coffman conditions present:

Condition	Present in Scenario
Mutual Exclusion	Yes — A, B, C are locked exclusively by one transaction
Hold and Wait	Yes — each Ti holds one resource while requesting another
No Preemption	Yes — locks are not forcibly released
Circular Wait	Yes — T1→T2→T3→T1 cycle

5. Justification

Recommended technique: Deadlock Prevention via resource ordering (breaks Circular Wait). Assign a fixed global order A < B < C. Every transaction must request locks in increasing order; a transaction needing an earlier resource after holding a later one must release and reacquire. This removes the possibility of a cycle since no transaction can hold a higher-ordered resource while requesting a lower-ordered one.

Final Answer

Redesign: Enforce locking order A → B → C for all transactions (T1: A then B; T2: B then C; T3: C then A must become C then, if A needed, release C first / acquire A before C). Alternatively, require transactions to acquire all needed locks atomically at the start (eliminating Hold-and-Wait). This structural fix prevents recurrence of the deadlock.

Question 3 — Mobile OS App Switching Lag

1. Understand the Scenario

Round Robin scheduling, quantum = 20 ms, 20 background apps active. Foreground interactive apps (WhatsApp, music, social media) lag; CPU shows high idle time despite many active processes.

2. Identify the OS Concept

Concept: Round Robin time-quantum sizing versus mixed CPU-bound/I/O-bound process behaviour in an interactive (mobile) OS.

3. Analysis

• High CPU idle time despite many processes → most background apps are I/O-bound (waiting on network/sensors/disk), not actually consuming CPU when scheduled; the CPU sits idle during their I/O waits instead of being handed to a ready interactive app.

• With 20 processes and a 20 ms quantum, one full round-robin cycle takes up to 20 × 20 ms = 400 ms in the worst case before an interactive app (e.g., WhatsApp) gets the CPU again.

• A fixed 20 ms quantum applied equally to background and foreground apps is too large and not priority-aware — it is hurting interactive performance.

4. Step-by-Step Solution

Aspect	Current (RR, Q=20ms, n=20)	Effect
Worst-case wait for foreground app	≈ 400 ms	Perceptible lag on app switch
CPU during I/O-bound app's turn	Idle if app blocks on I/O	Explains high idle time

Recommendation: Replace flat Round Robin with a Multilevel Feedback Queue (MLFQ) — foreground/interactive app gets a high-priority queue with a short quantum (≈5 ms); background/I/O-bound apps sit in lower-priority queues serviced when the foreground queue is empty.

5. Justification

• Short quantum for the foreground queue lets the OS return control to the active app within a few ms, removing switching lag.

• I/O-bound background apps naturally yield the CPU when blocked, so they don't need large quanta — MLFQ services them without wasting foreground responsiveness.

• Overall throughput improves because CPU is handed to whichever app is actually ready to run, instead of idling through fixed-length background slices.

Final Answer

Switch from flat Round Robin (Q=20ms) to a Multilevel Feedback Queue with a short quantum (~5 ms) for the foreground app and lower priority for background apps. This reduces app-switch lag and improves throughput by matching CPU allocation to actual process behaviour (interactive vs I/O-bound).

Question 4 — Deadlock Risk in an Operating System Update

1. Understand the Scenario

M1 holds R1, waits for R2. M2 holds R2, waits for R3. M3 holds R3, waits for R1. M4 (validate system integrity) runs independently. Resources: R1=Disk I/O, R2=Kernel lock, R3=Configuration lock.

2. Identify the OS Concept

Concept: Deadlock detection through circular wait among M1, M2, M3.

3. Analysis

```
M1 --> R2      (M1 holds R1, waits R2)
^        |
|        v
R1       M2 --> R3   (M2 holds R2, waits R3)
^               |
|               v
M3 <------------ R3   (M3 holds R3, waits R1)

Cycle: M1 -> R2 -> M2 -> R3 -> M3 -> R1 -> M1
```

4. Step-by-Step Solution

Yes, this sequence leads to deadlock. The circular wait is: M1 waits on R2 (held by M2); M2 waits on R3 (held by M3); M3 waits on R1 (held by M1). Since each resource is single-instance and locked exclusively, this closed chain of waiting modules can never proceed — a deadlock.

Banker's-algorithm-style avoidance: Before granting R1/R2/R3 to any module, the OS declares each module's maximum claim (e.g., M1 needs {R1,R2}, M2 needs {R2,R3}, M3 needs {R3,R1}) and runs the Safety Algorithm — a request is granted only if the resulting allocation state still leaves a safe sequence in which all modules can finish. A request that would create the R1→R2→R3→R1 cycle is simply refused/queued until safe.

5. Justification

Banker's algorithm avoids the deadlock proactively (before it occurs) rather than needing detection/rollback after modules are already stuck mid-update — important since a stalled OS update can corrupt system state.

Final Answer

Module prioritization: OS designers should sequence update modules by dependency instead of parallel free-for-all locking — e.g., grant Disk I/O (R1) → Kernel lock (R2) → Configuration lock (R3) in a fixed acquire order, run M4 (validation) only after M1–M3 release their locks. This removes circular wait, keeps CPU utilization high (modules run without blocking each other), and prevents deadlock recurrence.

Question 5 — Cloud Server CPU Utilization Analysis

1. Understand the Scenario

Each VM process spends 80% of time waiting for I/O (p = 0.8) and 20% on CPU. A server runs n = 7 VMs concurrently.

2. Identify the OS Concept

Concept: CPU Utilization under multiprogramming, using CPU Utilization = 1 − p^n (probability that at least one of n processes is CPU-ready).

3. Analysis

Formula: CPU Utilization = 1 − p^n, where p = I/O wait probability, n = number of concurrent VMs.

4. Step-by-Step Solution

```
CPU Utilization = 1 - p^n
                = 1 - (0.8)^7

(0.8)^7 = 0.2097152

CPU Utilization = 1 - 0.2097152
                = 0.7902848
                ≈ 79.03%
```

5. Justification

• Even though each individual VM is idle (I/O-waiting) 80% of the time, running 7 VMs concurrently means the probability that ALL 7 wait simultaneously is only ≈20.97%, so the CPU stays busy ≈79% of the time.

• This is close to well-utilized — the server is not underutilized; multiprogramming is doing its job.

• [object Object],[object Object],[object Object],[object Object]

Final Answer

Item	Value
CPU Utilization	≈ 79.03%
Server Status	Well-loaded (not underutilized)

Two OS-level strategies to further improve utilization: (1) Asynchronous/non-blocking I/O so a VM's CPU-ready work can proceed while its I/O request is in flight, instead of the vCPU sitting idle; (2) Increase the multiprogramming degree with an I/O-aware scheduler (e.g., favor I/O-bound VMs for quick dispatch, CPU-bound VMs fill remaining idle slots) so more VMs' compute bursts overlap other VMs' I/O waits.

Question 6 — Hospital Emergency System Process Scheduling

1. Understand the Scenario

P1 (emergency registration, highest priority), P2 (lab report generation, medium), P3 (pharmacy billing, low). During peak hours P3 keeps getting delayed as emergency requests (P1) keep arriving.

2. Identify the OS Concept

Concept: Starvation under strict Priority Scheduling — a continuous stream of high-priority arrivals indefinitely postpones the low-priority process.

3. Analysis

The scheduling issue is starvation of P3 (pharmacy billing). Because P1 keeps arriving and always preempts, and P2 also outranks P3, the CPU never reaches P3 as long as higher-priority work exists in the ready queue.

4. Step-by-Step Solution

Recommended algorithm: Preemptive Priority Scheduling with Aging.

Process	Priority	Aging Rule
P1	Highest	Fixed — always preempts
P2	Medium	Priority raised gradually if it waits too long
P3	Low	Priority increased by a fixed step for every fixed wait interval (e.g., +1 every 5 ms of waiting) until it matches/exceeds P2/P1

How starvation is prevented: Aging continuously raises the waiting process's effective priority. Once P3's aged priority reaches the level of P1/P2, the scheduler is forced to dispatch it, guaranteeing an upper bound on its wait time regardless of how many emergency requests keep arriving.

5. Justification

• Emergency registration (P1) still gets immediate service — patient safety is not compromised.

• Aging gives pharmacy billing (P3) a guaranteed maximum response time instead of indefinite postponement — improves fairness.

• Response time for P1 stays near-zero (true real-time need), while P2/P3 get bounded, predictable turnaround instead of open-ended delay.

Final Answer

Use Preemptive Priority Scheduling with Aging: P1 keeps top priority for real emergencies, but P2 and P3 gain priority the longer they wait, so pharmacy billing is guaranteed to run within a bounded time — fixing starvation while preserving emergency responsiveness.

Question 7 — E-Commerce Warehouse Synchronization

1. Understand the Scenario

Multiple worker threads update a shared inventory database simultaneously, causing incorrect stock counts, duplicate order confirmations, and data inconsistencies.

2. Identify the OS Concept

Concept: Race condition — concurrent threads perform unsynchronized read-modify-write on shared inventory data (the critical section).

3. Analysis

The stock-count variable and order-confirmation flag are shared resources. Without synchronization, two threads can interleave their read → modify → write steps, so one thread's update is lost or an order is confirmed twice. This is precisely the classic critical-section problem.

Why a critical section is required: Any code segment that reads and updates the shared inventory record must execute atomically (as one indivisible unit) so no other thread can interleave — this segment is the critical section and must be protected.

4. Step-by-Step Solution

Recommended mechanism: Binary Semaphore (Mutex), initialized to 1, wrapping the inventory-update critical section.

```
semaphore mutex = 1;

UpdateStock(item, qty) {
    wait(mutex);        // enter critical section
    read stock[item];
    stock[item] -= qty;
    write stock[item];
    confirm_order();
    signal(mutex);      // exit critical section
}
```

5. Justification

• wait(mutex) decrements the semaphore; if it is already 0, the calling thread blocks — only one thread can be inside the critical section at any time (mutual exclusion).

• signal(mutex) releases the lock after the update completes, letting the next waiting thread enter — guarantees progress, no busy-waiting on a spinlock.

• Because the read-modify-write on stock[item] and the order-confirmation happen as one atomic block, no thread can see or act on a stale/intermediate value → eliminates incorrect stock counts and duplicate confirmations.

Final Answer

Wrap the shared inventory update in a binary semaphore (mutex) so only one thread executes the critical section at a time. This enforces mutual exclusion, removes the race condition, and keeps stock counts and order confirmations consistent.

Question 8 — Video Streaming Service Thread Management

1. Understand the Scenario

One thread is created per client connection. During a live event, 60,000 simultaneous users push CPU utilisation to 95%, memory usage rises rapidly, and video buffering becomes frequent.

2. Identify the OS Concept

Concept: Thread-per-connection explosion — the One-to-One threading model does not scale: 60,000 kernel threads each carry their own stack, context, and scheduling overhead.

3. Analysis

Symptom	Cause
Memory rising rapidly	Each of 60,000 threads holds its own stack + kernel thread-control-block
CPU at 95% but buffering	Excess time spent on context switching between threads, not on encoding/serving video

4. Step-by-Step Solution

Recommended model: Many-to-Many (M:N) threading model — map a large number of user-level client tasks onto a small, bounded pool of kernel threads.

Two optimisations:

• Fixed-size worker thread pool with a task/connection queue — reuse a bounded set of kernel threads instead of spawning one per client, capping context-switch and memory overhead.

• Event-driven / asynchronous I/O (e.g., epoll-based reactor) so a single worker thread can monitor thousands of idle/waiting connections and only actively process the ones with data ready.

5. Justification

• Bounded thread pool → memory usage stays proportional to pool size, not connection count.

• Fewer kernel threads → far fewer context switches → CPU cycles go to actual frame processing instead of scheduling overhead.

• Async I/O lets one thread service many idle connections cheaply, so more CPU capacity is left for active video streams → less buffering.

Final Answer

Move from thread-per-client to a Many-to-Many model built on a bounded worker-thread pool plus event-driven asynchronous I/O. This caps memory growth, cuts context-switching overhead, and frees CPU cycles for actual streaming work — improving throughput and reducing buffering.

Question 9 — Smart Traffic Control Deadlock

1. Understand the Scenario

T1 holds R1(North), requests R2(East). T2 holds R2, requests R3(South). T3 holds R3, requests R4(West). T4 holds R4, requests R1.

2. Identify the OS Concept

Concept: Deadlock via a 4-way circular wait among intersection controllers.

3. Analysis — Resource Allocation Graph

```
T1 --> R2    (T1 holds R1, requests R2)
^        |
|        v
R1       T2 --> R3   (T2 holds R2, requests R3)
^                |
|                v
T4 <-- R4 <----- T3   (T3 holds R3, requests R4; T4 holds R4, requests R1)

Cycle: T1 -> R2 -> T2 -> R3 -> T3 -> R4 -> T4 -> R1 -> T1
```

4. Step-by-Step Solution

1. Deadlock exists: Yes — a closed cycle links all four single-instance junction resources; none of the four controllers can proceed.

Condition	How it Applies Here
Mutual Exclusion	Each junction (R1–R4) is controlled by only one traffic controller at a time
Hold and Wait	Each Ti holds its current junction while requesting the next one
No Preemption	A junction's green-light control cannot be forcibly taken from the holding controller
Circular Wait	T1→T2→T3→T4→T1 forms a closed chain of requests

5. Justification

Recommended recovery: Resource Preemption — the central traffic coordinator forcibly preempts one junction resource (e.g., overrides T4 and reassigns R4), breaking the cycle. T4 is rolled back to re-request R4 afterward. Preemption is preferred over killing a controller process, since aborting a traffic controller is unsafe — vehicles would be left uncontrolled at that junction.

Final Answer

Redesign to prevent recurrence: Introduce a central coordinator that grants junction requests only via a Banker's-style safety check (or a fixed acquisition order R1<R2<R3<R4) before allocation, so a request that would close a cycle is queued instead of granted. This converts the system from reactive recovery to proactive deadlock avoidance.

Question 10 — University Examination Server Resource Allocation

1. Understand the Scenario

Thousands of students log in simultaneously to an exam server allocating CPU, memory, and DB connections; new requests arrive continuously. The administrator wants to avoid deadlock without reducing resource utilisation.

2. Identify the OS Concept

Concept: Deadlock Avoidance using Banker's Algorithm.

3. Analysis

Each student session is a process with a declared maximum resource claim (CPU slice, memory, DB connections). Banker's Algorithm grants a new request only if the resulting state is still 'safe' — i.e., some order exists in which all sessions can finish with the available resources.

4. Step-by-Step Solution

1. Applying Banker's Algorithm: On every resource request from a student session, the server tentatively grants it, then runs the Safety Algorithm before committing; if unsafe, the request is denied/queued and the session waits, otherwise it is confirmed.

2. Information required before granting a request:

Structure	Meaning
Available	Vector of currently free CPU/memory/DB-connection units
Max	Matrix of each session's maximum possible claim per resource type
Allocation	Matrix of resources currently held by each session
Need	Need = Max − Allocation, per session

3. Determining a safe state (Safety Algorithm): Find a session whose Need ≤ Available; assume it runs to completion and releases its Allocation back into Available; repeat for the remaining sessions. If all sessions can be finished this way, the state is safe; if at some point no session's Need fits Available, the state is unsafe and the pending request is refused.

5. Justification

Banker's Algorithm avoids deadlock proactively while still granting requests whenever safe, so resource utilisation is not needlessly reduced — matching the administrator's requirement.

Final Answer

Limitation in large-scale cloud systems: Banker's Algorithm requires every session to pre-declare a fixed Max claim, which is unrealistic for thousands of dynamically-arriving, short-lived student sessions with unpredictable resource needs; also, the Safety Algorithm's cost grows with the number of processes and resource types ($O(n^2 \cdot m)$ per check), so re-running it on every request at exam-time scale adds significant latency and does not suit elastic, auto-scaling cloud environments.

RUBRICS FOR CALCULATION

Scenario based assessment

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding ; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

Marks Evaluation:

Question No.	Understanding of Scenario (20%)	Identification of Key Issues (20%)	Application of Concepts (25%)	Decision Making (20%)	Clarity of Response (15%)	Total Marks(100)
Q1						
Q2						
Q3						
Q4						
Q5						
Q6						
Q7						
Q8						
Q9						
Q10						
					Overall Total (100)	

Faculty In-charge	Course Coordinator	Head of Department
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____